

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order.*

Example 1:

Input: `nums = [-4,-1,0,3,10]`

Output: `[0,1,9,16,100]`

Explanation: After squaring, the array becomes `[16,1,0,9,100]`.

After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`

Output: `[4,9,9,49,121]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in **non-decreasing** order.

Program

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& nums) {
        int n = nums.size();
        vector<int> result(n);

        int left = 0;
        int right = n - 1;

        for (int i = n - 1; i >= 0; i--) {
            if (abs(nums[left]) > abs(nums[right])) {
```

```
        result[i] = nums[left] * nums[left];  
        left++;  
    } else {  
        result[i] = nums[right] * nums[right];  
        right--;  
    }  
}  
  
return result;  
}  
};
```

Output :

☒ Testcase  Test Result

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

```
nums =  
[-4, -1, 0, 3, 10]
```

Output

```
[0, 1, 9, 16, 100]
```

Expected

```
[0, 1, 9, 16, 100]
```

